Operating Systems

NAME: Alisha Kulshrestha
SRN: PES1UG24CS049
SECTION: A
DATE: 20/04/2026

PHASE 1

```
[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

```
[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

PHASE 2

```
[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

```
[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$ find .pes/objects -type f
.pes/objects/d7/1b25693891507a11cd5dbc58f431b16b1ec9a907dafafac66014e1585d775e
[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$ xxd .pes/objects/d7/1b25693891507a11cd5dbc58f431b16b1ec9a907dafafac66014e1585d775e
00000000: 7472 6565 2034 3900 3130 3036 3434 2064  tree 49.100644 d
00000010: 6562 7567 2e74 7874 00ab abab abab abab  ebug.txt........
00000020: abab abab abab abab abab abab abab abab  ................
00000030: abab abab abab abab ab                  .........
[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$
```

PHASE 3

```
[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.h
  untracked:  Makefile
  untracked:  index.h
  untracked:  test.txt
  untracked:  tree.c
  untracked:  test_sequence.sh
  untracked:  object.c
  untracked:  commit.c
  untracked:  index.c
  untracked:  dummy.c
  untracked:  force_add.c
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  .gitignore
  untracked:  debug.txt
  untracked:  safe_add.c
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  safe_add
  untracked:  commit.h
  untracked:  data.txt
  untracked:  force_add
  untracked:  screenshot_file.txt
  untracked:  pes.h
```

```
[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$ cat .pes/index
100664 a1877c76445e320ef3762ddeb5845a4e9d799f7aa0c1312da032ba2d3e54ce12 1776693410 9 file1.txt
100664 bacf08269a13c76f8b14fbc948bd577ca5a294f0059b3dfdd8cf52a0ad8aaa7a 1776693410 9 file2.txt
[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$
```

PHASE 4

```
[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$ ./pes log
commit 0213789e684c9991e5be149c9be953be780624538213456b9bff0f345fdd1b11
Author: Alisha Kulshrestha <PES1UG24CS049>
Date:   1776694723

    Initial commit

[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$
```

```
[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/02/13789e684c9991e5be149c9be953be780624538213456b9bff0f345fdd1b11
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/50/da219c4c66a82940f7a8555908504a9bb1ede3773efe8669ad72f1d54966b3
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/67/983e9aac9901b16895a1a3082ef00d6dc79894abc18c41a86dfcfa9d8ce1f7
.pes/objects/7c/83d6df2d741034233b181c4bfa7a6e64a49ec875cc62a850886d11af72429c
.pes/objects/82/e992c120b328608f30499445fee5dd3fb31abe7e7c1f89975546f270106cb7
.pes/objects/c1/5daab6ccf47288c923fa5384e07d0e7c63a4f443a1a77c77d51e67b257e822
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$
```

```
[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$ cat .pes/refs/heads/main
82e992c120b328608f30499445fee5dd3fb31abe7e7c1f89975546f270106cb7
[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
[04/20/26]seed@VM:~/PES1UG24CS049-pes-vcs$
```

PHASE 5

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Implementing checkout requires updating .pes/HEAD to the target branch reference and synchronizing the working directory with the branch's tree. It is complex because it must safely delete files not present in the new branch while preventing the loss of uncommitted local changes.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

A "dirty" directory is detected by comparing the mtime or hash of files on disk against the index. If a file is modified but not committed, and that file differs in the target branch, the checkout must refuse to proceed to avoid overwriting the user's unsaved work.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Garbage Collection and Space Reclamation

In a "Detached HEAD" state, new commits are created normally, but HEAD stores the hash directly instead of a branch pointer. If the user switches branches, these commits become "orphaned" and can only be recovered by finding their hashes in the object store or reflog to create a new branch.

PHASE 6

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

To delete unreachable objects, a Mark and Sweep algorithm is used, starting from branch tips and HEAD to mark all reachable hashes in a Hash Set. For 100,000 commits and 50 branches, the system would visit at least 100,000 commit objects plus all unique trees and blobs to determine reachability.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Concurrent GC is dangerous because it could delete a newly written blob before the commit object links it to a branch, leading to repository corruption. Git avoids this race condition by using a grace period, only pruning unreachable objects that haven't been modified for a set time (e.g., 14 days).